

Source-Filter Vowel Resynthesis Across English Sentences

According to the Source-Filter Theory, a vowel is made up of a source, or voicing from the larynx, and a filter, made up of the rest of the vocal tract and positioning of the mouth. Each vowel has different frequencies that are most resonant in the cavity of the vocal tract—these are its formants. One way to model the shape of the filter and find the formants is using linear predictive coding (LPC). This process begins with framing, which is breaking the signal down into small chunks. Next, an autocorrelation function is used to generate coefficients that describe the spectral envelope of the filter (Styler, 2024, “Linear Predictive Coding”).

By changing the LPC of a vowel but keeping the source the same, a vowel sound can be manipulated into a completely different one. In my project, my goal is to work towards accent manipulation of recorded speech based on the vowel change patterns of different English dialects. I created a Python function that uses LPC resynthesis through Praat script to change every instance of a prespecified monophthongal vowel in a sentence to another prespecified vowel, i.e. changing every /ε/ into an /ɪ/. Of course, vowel changes are not the only thing that differentiates accents and the vowels are not always changed in every environment systematically. There may be differences in consonant productions, rhoticity, prosody, vowel length, etc. This project will only focus on changing the formants of the existing vowels. In this paper, I first introduce the existing resources I used, and what the algorithm does step by step. Then I test the algorithm with sentences and discuss the results as well as next steps for the future.

Resources

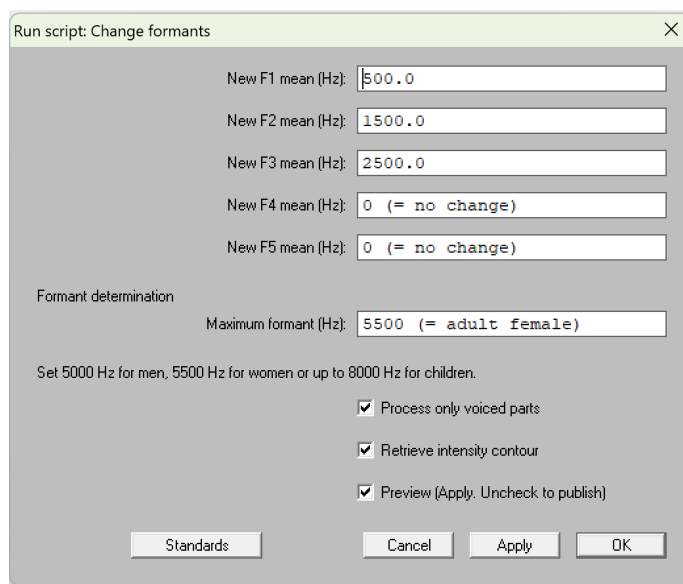
I used Google Colaboratory (Colab) to write and run my Python code. A link to the notebook is on the cover page, along with the GitHub containing all the files I used.

Praat Vocal Toolkit

The main tool I will utilize is the “Change Formants” Praat script from Ramon Corretge’s Praat Vocal Toolkit, a Praat plugin containing many scripts for a variety of processing tasks. The `changeformants.praat` script allows the user to specify the target mean values for F1, F2, F3 (and F4/F5 if desired) for the new vowel. Additionally, the user can enter a maximum formant value based on the voice used, and select a boolean value for “Process only voiced parts” and “Retrieve intensity contour” (See Figure 1).

Figure 1

Variables Prompted by Form in changeformants.praat Script in Praat



Run script: Change formants

New F1 mean (Hz): 500.0

New F2 mean (Hz): 1500.0

New F3 mean (Hz): 2500.0

New F4 mean (Hz): 0 (= no change)

New F5 mean (Hz): 0 (= no change)

Formant determination

Maximum formant (Hz): 5500 (= adult female)

Set 5000 Hz for men, 5500 Hz for women or up to 8000 Hz for children.

☒ Process only voiced parts

☒ Retrieve intensity contour

☒ Preview (Apply. Uncheck to publish)

Standards Cancel Apply OK

Charsiu

In order to recognize and identify the vowels in a sentence, I will use the ASR tool Charsiu made by Jian Zhu, Cong Zhang, and David Jurgens. Charsiu’s textless alignment creates a list of tuples, each tuple containing the starting and ending times of each phone in seconds and its phone label as a string. The phone labels use ARPAbet transcriptions, where each sound

corresponds to a two-letter code. I will use the labels to determine which vowels need to be modified.

Python Libraries

In addition to the libraries needed to run Charsiu, I also installed libraries that make working with audio easier. Parselmouth is used to run Praat commands and scripts from Python. I could not find many examples of how to use Parselmouth commands, so one of the challenges was understanding the API reference. Additionally, any commands in Parselmouth can only be done to a Sound object. This was inconvenient for me for two reasons: Firstly, I wanted the input of my vowel change function to be a path to a wav file. Secondly, I could not find a way to play a Parselmouth Sound object directly in Python without going through yet another conversion, or writing over the original file. Thus, I chose to install the pydub AudioSegment library since an AudioSegment object can be played directly.

Implementation

The first portion of code in the notebook is installing the necessary libraries mentioned above. I initialize the Charsiu textless alignment model so it can be used in the main function. I then connect to my Google Drive so it can access my test audio files. The main function, **replace_vowels()**, takes three string arguments in the following order: a path to a wav file (containing a spoken sentence), the vowel that the user wants to change in ARPAbet code, and the new vowel to change to in ARPAbet code. It then performs the following steps:

1. Textless alignment on the given wav file with Charsiu. This is done using a previously defined function called **textless_align()**. It requires resampling the sound to 16,000 Hz first. I also print the alignment tuple so I can see if there were any transcription errors.
2. Initialize a Parselmouth Sound object with the wav file.

3. Downsample the Sound object to 16,000 Hz. According to Styler (2023), “resynthesis works best with downsampled sounds” (p. 49). I tried several frequencies, but 16000 sounded the best to me.
4. Go through alignment labels to find the vowel to change. The third item in every tuple created by Chariu textless alignment contains the ARPAbet label of the phone. Each iteration checks if the label of the phone matches the vowel that the user wants to change. If it doesn’t match, nothing happens and it moves on to the next label.
5. If the label is found, extract the segment with the vowel. Using Parselmouth’s **extract_part()** method, a new Sound object is created using the starting and ending times of the phone. This is the vowel which will have its formants modified.
6. Run `changeformants.praat` on the extracted vowel. The Praat script is run by the **run_file()** function in Parselmouth, with the same arguments that the form asks for in Figure 1. Generally, the script does source-filter resynthesis of vowels similarly to what Will Styler details in section 8.15 of *Using Praat for Linguistic Research*. It creates an LPC object from the vowel with **To LPC**. Using **Filter (inverse)** on the vowel and LPC object creates “the ‘neutral’ source voicing from that vowel” which is maintained throughout the resynthesis, as the only thing we want to change is the filter (Styler, 2023, p. 48). Next, the script creates a formant object from the vowel with **To Formant (robust)**. The mean values for formants 1 through 5 are calculated to find the difference between the mean formants and the target mean formants input by the user. The differences are the “df” variables shown in Figure 2. The formants are then modified using formula. Instead of raising the formant by a fixed number of Hertz, each formant raised by however much the difference is between the target formant value and the mean

formant value. Finally, the source voicing is combined with the modified LPC to create the output sound. I am using 5000Hz as the maximum formant as I will be testing with a male adult voice. For the F1, F2, and F3 values for each monophthongal vowel, I am using the values from UCLA professor Bruce Hayes' Vowel Chart with Sound Files. These formants were calculated with an adult male voice similar to the TTS voice that I will test with. Depending on what string was inputted as the new desired vowel, the script will run with the formant values that match the vowel. Since the ARPAbet only has codes for vowels present in the "general American" accent, in which the vowel /e/ typically doesn't occur on its own, I had to create my own code for /e/ ("EE") for one of my tests.

Figure 2

Changing Formants by Formula in Praat Script

```
if f1 <> 0 and abs(df1) < 1000
    Formula (frequencies): "if row = 1 then self + df1 else self fi"
endif
if f2 <> 0 and abs(df2) < 2500
    Formula (frequencies): "if row = 2 then self + df2 else self fi"
endif
if f3 <> 0 and abs(df3) < 2500
    Formula (frequencies): "if row = 3 then self + df3 else self fi"
endif
if f4 <> 0 and abs(df4) < 2500
    Formula (frequencies): "if row = 4 then self + df4 else self fi"
endif
if f5 <> 0 and abs(df5) < 2500
    Formula (frequencies): "if row = 5 then self + df5 else self fi"
endif
```

7. Put the modified vowel back into the original sentence. Parselmouth's **concatenate()** method stitches together three parts: The first chunk is the beginning of the file to the start of the changed vowel, the second chunk is the changed vowel, and the third chunk is the end of the changed vowel to the end of the audio (its total duration).
8. Repeat steps 3-6 until all the labels are checked and every instance of vowel to change is modified.

9. Upsample to 44,100Hz
10. Turn the whole sentence, currently a Parselmouth Sound object, into an AudioSegment object.
11. Return the AudioSegment object.

Testing and Results

I tested the algorithm with Google's web-based text-to-speech male English voice. TTS voices are, in general, fairly limited in their range of accents and dialects. Diversifying TTS voices has important sociolinguistic implications and may affect how we think of its role or positionality. Thus, my goal is oriented towards expanding the accents of TTS voices. I focused on having the function perform vowel shifts that are common in different accents of English.

Shift from /ɛ/ (EH) to /ɪ/ (IH)

Some Southern U.S. speakers pronounce "pen" and "pin" the same way, where the mid-low front unrounded vowel /ɛ/ is shifted closer to an /ɪ/, especially before a nasal consonant. I replicated this change with the sentences "When I win, I will get a pen" and "How many hens are in the den?" to test the function's ability to turn every instance of /ɛ/ ("EH" in ARPAbet) to /ɪ/ ("IH"). Each test file contains Google's TTS saying the sentence at 0.55 speaking speed. I ran the **replace_vowels()** function on the files and also played the original test files for comparison. The outputs can be played on the Colab (not GitHub copy) notebook linked on the cover page.

The output of the first sentence was pretty good, although there was a pop on "when." The vowel change is not very noticeable on "when" and "get," but then "pen" definitely sounds more like "pin" to my ears, which was the goal. On "How many hens are in the den," the "hens" was not changed at all. This is because the Charsiu alignment transcribed the vowel in "hens" as

/æ/, and thus the function didn't detect it as a vowel to change. "Den" did sound more like "din," similarly to "pen."

Shift from /æ/ (AE) to /e/ (EE)

In the Great Lakes region of the U.S. (e.g. cities near Chicago), the Northern Cities Vowel Shift changes the /æ/ vowel to the /e/ vowel for some speakers (WIRED, 2021). My first test sentence, "Put the cat in the bag," didn't sound so great. "Bag" still sounds like /bæg/ but with extra distortion. My guess as to why this happened was that Charsiu didn't capture the whole /æ/ vowel in its time boundaries. I also tried "The bag is full of cats," and the results were much better. This may be because the /æ/ vowel was longer and more emphasized in this second sentence, while it was shorter when "bags" was placed at the end of the sentence.

Shift from /u/ (UW) to /ʌ/ (AH)

Another vowel change I tested was the /u/ vowel shifted to /ʊ/ or /ʌ/, particularly in the word "room." This is common in some New England accents, but can be heard in various parts of the country. I tested this with "There is a pair of boots in the room." I don't know if there are speakers who say /bʊts/ for "boots," but I wanted two instances of the vowel to see how the function worked on both instances. Originally, I ran the function with /ʊ/ as the new vowel, but the results were very strange. The vowel in "boots" was extremely distorted and barely intelligible. I tried again with /ʌ/ as the new vowel, and this worked better for the word "room." However, "boots" turned into something like /bʊts/. This is interesting because "boots" and "room" both have a /u/ vowel, yet when the same function was run with the same target formants, the resynthesized vowels sounded drastically different.

Discussion and Conclusion

Overall, my function worked, although not to the degree that I had hoped. I had anticipated this, because LPC resynthesis is a difficult task and it usually doesn't end up sounding very natural. Even when the vowel change was noticeable with my function, it still sounded quite distorted or with extra artifacts like clicks and pops. The best performances in my opinion were the /ɛ/ to /ɪ/ change in "pen" and the /æ/ to /e/ change in "bag."

I hypothesize that one of the biggest problems contributing to the mediocre performance is that the Charsiu aligner does not get a precise cutoff of the vowel boundaries. The starting and ending times are to the nearest hundredth of a second, so the boundaries may include some of the neighboring sounds and the function ends up modifying more than just the vowel. Another problem may come from coarticulation. The consonants bordering a vowel may be produced slightly differently depending on what vowel is preceding or following. For example, the /d/ in "deed" is pronounced with the mouth configuration for an /i/. Thus, changing the /i/ to a different mouth configuration but not the /d/ gives unnatural results. This is why when I tried changing the /u/ vowel to /ʌ/ in "boots" and "room," the results were drastically different for the two vowels even though they are the same transcribed phoneme. Vowels in speech are also very different from vowels in isolation. Even vowels characterized as monophthongal might have slight shifts at its onset or end, making its formants vary across the vowel.

If I had more time, I would try to implement the formant changing in the Praat script formula by smaller increments instead of making the leap from the original to target formant in one go. Also, I only worked with monophthongal vowels so far because diphthongs contain two sounds and would require shifting the formants in the middle of the vowel. Consequently, I could not capture any diphthong changes in accents. This would be a more complex task and I am not sure how to approach it yet, but it could be a future topic of my study. To make even more

progress towards full accent manipulation, I would work on doing multiple vowel changes at once. As of now, the function only allows for one change at a time. I would also look into how to elongate vowels in certain environments that align with particular accents.

This project reinforced my understanding of several concepts we learned in class: resampling, automatic speech recognition, source-filter theory, LPC resynthesis, and text-to-speech. I also now understand a lot more about how Praat scripting works for processing speech and how to perform LPC resynthesis in detail. There is still a lot of potential for this project and I hope eventually it can contribute to expanding the accent diversity of TTS voices.

References

- Corretge, R. (2024). Praat Vocal Toolkit. <https://www.praatvocaltoolkit.com>
- Hayes, B. (2013). *Vowel Chart with Sound Files*. Linguistics 103 - vowel chart with sound files.
<https://linguistics.ucla.edu/people/hayes/103/Charts/VChart/>
- Jadoul, Y. (2022). *Praat in python, the Pythonic way*. Parselmouth.
<https://parselmouth.readthedocs.io/en/stable/>
- Styler, W. (2024, April 12). *Linear Predictive Coding*.
https://wstyler.ucsd.edu/talks/1168_6_lpc_handout.html#/
- Styler, W. (2023). Using Praat for Linguistic Research - Version 1.9.2.
<https://wstyler.ucsd.edu/praat/>
- Vertex AI. (2024). *Speech - Vertex AI - Google Cloud console*. Google Cloud.
<https://console.cloud.google.com/vertex-ai/generative/speech/text-to-speech?project=fit-a-loe-423503-r3>
- WIRED, & Singer, E. (2021, February 17). *Accent Expert Gives a Tour of U.S. Accents - (Part 2)*
| *WIRED*. [Video]. YouTube. https://youtu.be/IsE_8j5RL3k?t=450
- Zhu, J., Zhang, C., & Jurgens, D. (2022). *Charsiu: A neural phonetic aligner*. GitHub.
<https://github.com/lingjzhu/charsiu/>